PITTACUS: Optimizing LLM Serving on Cloud Data Centers with Forecast Aware Auto-Scaling

Anonymous Author(s)

Abstract

Inference workloads for Large Language Models (LLMs) span both latency-sensitive (for e.g. chatbots) and insensitive (for e.g. report writing) tasks, resulting in diverse and often conflicting Service Level Agreement (SLA) requirements. Managing such mixed workloads is challenging due to the complexity of the inference stack which encompasses multiple models, hardware types, and globally distributed data centers. Existing solutions often silo tasks to meet SLAs, but this leads to significant under-utilization of expensive GPU resources. We propose PITTACUS, a comprehensive LLM serving framework that dynamically adapts to workload demands using multi-timescale control knobs. PITTACUS combines short-term request routing with long-term GPU VM scaling and model placement, formulating the resource allocation challenge as an Integer Linear Programming (ILP) problem. Through real and simulated evaluations on 8 million production requests across three regions and four open-source models, PITTACUS achieves up to 25% savings in GPU-hours, reduces scaling overhead by 80%, offering potential monthly cost savings of up to $2 million for cloud providers, while maintaining tail latency and satisfying all SLOs.

ACM Reference Format:

Anonymous Author(s). 2025. PITTACUS: Optimizing LLM Serving on Cloud Data Centers with Forecast Aware Auto-Scaling. In . ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Recent years have seen rapid adoption of *Large Language Models (LLMs)* in enterprise products and services to power both proactive and user-initiated intelligent features [6]. As their capabilities expand, LLM usage is growing exponentially across enterprise, consumer, and scientific applications—including accelerating scientific reasoning and discovery [8, 10, 21].

Motivation. Cloud-hosted, GPU-accelerated Virtual Machines (VMs) are central to scaling LLM inference, prompting major investments from Cloud Service Providers (CSPs) for both internal use and public offerings. AWS UltraClusters offer 100,000 of Trainium2 accelerators used by Anthropic and end-users [1] while Microsoft Azure Cloud's Eagle system with H100 GPUs features at #4 in the Top500 list [2]. HPC machines with GPU accelerators are also leveraging AI models and LLMs, complementing their traditional use for simulations [7, 12]. However, maximizing return on these expensive resources is critical. Misalignment between GPU provisioning and traffic distribution across regions can lead to SLA violations or resource wastage at either extreme. The need to meet SLAs and provide a smooth user experience generally causes CSPs to overprovision GPU capacity, raising infrastructure costs, increasing prices for users, and diverting resources from R&D.

Challenges. Unlike well-studied CPU VM auto-scaling [15, 34], scaling GPU VMs for LLM workloads presents unique challenges. Commerical platforms like Gemini [3], Copilot [2], and ChatGPT [1] serve a mix of models and inference workloads that can broadly be categorized as a) *Interactive workload (IW)* that are latency-sensitive and require real-time responses, within milliseconds or seconds (for e.g. chatbots, LLM powered search, content moderation), and b) *Non-interactive Workload (NIW)* requests are less time-critical and focus on serving resource-intensive or batch processes (for e.g.. report writing, data annotation, and simulations). These workloads vary by time, region, and user type (enterprise/consumer), making it difficult to design a unified auto-scaling policy that efficiently handles diverse models and SLA requirements.

Scaling LLM model instances can be both ineffective, due to traffic variations, and slow, blocking GPUs for many seconds or minutes, due to the large size of LLMs (for e.g. Meta's Llama-2-70B model [35] is around 140GB in float16, and OpenAI's GPT models are estimated to be even larger). Reactive scaling based on metrics like incoming *Tokens Processed per Second (TPS)*) can cause over- or under-provisioning, as it fails to account for TPS variance and LLM loading delays. For e.g. in Fig. 1, the model instance has a capacity to serve 4000 TPS. In the top plot the reactive strategy decides to scale up the instance count at $T = 15$ mins (top), which causes the instance to be available at $T = 20$ mins, resulting in SLA violations for 5 mins due to under-allocation. If we consider a conservative instance capacity of 3500 TPS to avoid this, the reactive approach is susceptible to over-allocation as seen in the bottom plot where the reactive strategy unecessarilly scales up at $T = 25$ mins due to a small increase in traffic even though the incoming TPS stabilizes. There is a pressing need for a flexible, lightweight auto-scaling policy that adapts to dynamic workloads, minimizes model loading delays, and reduces costs while meeting diverse SLAs.

Approach. We present PITTACUS, informed by real-world LLM workloads at a major CLOUD PROVIDER X serving LLM requests across global data centers (regions). The currently deployed *siloed approach*, with separate GPU pools for IW and NIW requests, is seen to significantly underutilize the IW resource pool in off-peak hours. PITTACUS first introduces a *reactive heuristic* over a unified model pool (Fig. 2b), routing NIW requests to underloaded

[1] https://www.hpcwire.com/2024/12/05/aws-delivers-the-ai-heat-project-rainier-and-genai-innovations-lead-the-way/, Dec 2024

[2] https://www.hpcwire.com/2024/11/18/the-captain-has-crossed-the-frontier/, Nov 2024

Conference'17, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

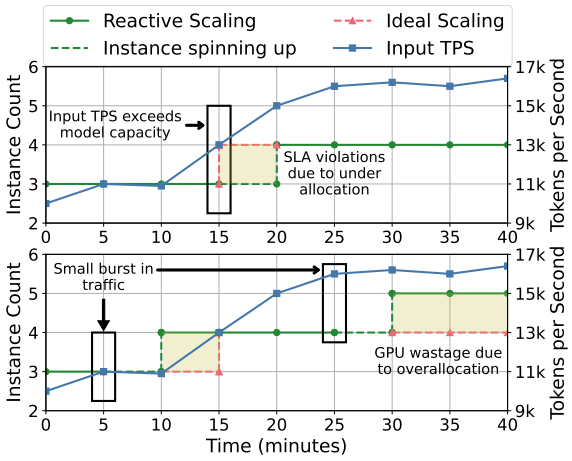
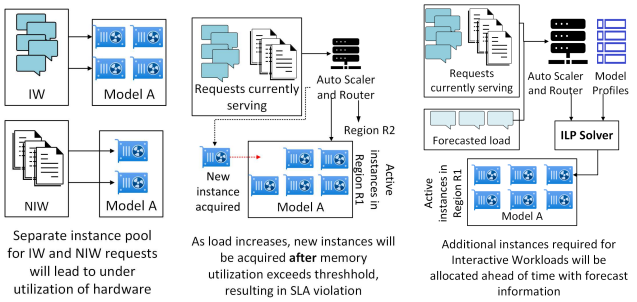


Figure 1: VM instance scaling (left Y axis) based on incoming TPS (right Y axis) using ideal (pink) and reactive (green) strategies. Shaded region shows the difference in their instance counts. *Top* shows reactive configured with a higher TPS limit for scaling-up, leading to SLA violations. Bottom shows a lower TPS for scaling, causing over allocation.

GPUs to save GPU hours while meeting SLAs. To address demand-supply mismatches across model types (e.g., GPT-4.1, Llama4-Scout, Mistral-Nemo), Pittacus then applies a *predictive heuristic* (Fig. 2c), formulating a constrained optimization problem for scaling LLM instances based on ARIMA-based traffic forecasts. This enables dynamic model scaling across regions, improves GPU utilization, ensures SLA compliance, and allows surplus capacity to be offered as spot instances.

We make the following *specific contributions* in this paper:

(1) We highlight the challenge of building LLM serving platforms for CSPs managing GPU VMs across regions with diverse workloads, motivating the need for holistic, continuous optimization of LLM instance placement to improve resource efficiency (§ 3).
(2) We present empirical studies on real-world workloads at Cloud Provider X that highlight the pitfalls of siloed decisions and motivate a systematic instance scaling approach (§ 4).
(3) We propose Pittacus, a unified framework for serving diverse LLM inference workloads across cloud regions, aiming to meet SLAs while maximizing GPU resource efficiency. Pittacus formulates the scaling of GPU VMs and model instances as an ILP-based optimization problem (§ 5), leveraging traffic forecasting models and considering practical factors such as instance capacity (in TPS), IW demand, NIW headroom, provisioning overheads, and surplus donation to spot services. (§ 6).
(4) We implement a prototype of Pittacus and evaluate it using a realistic simulation harness based on SplitWise [29]. We compare Pittacus with a State of the Art (SOTA) auto-scaler, *Chiron* [31], for a week-long real-world trace (10M requests, 3 regions, 4 LLMs). Pittacus reduces GPU VM usage by 25% through improved utilization and reduces scaling overhead by 80% without compromising any SLAs, which translates into savings of over US$2M per month. We will open-source our trace data and simulator to serve as a testbed for the community, as described in the *Artifact Description Appendix (AD)*.



(a) Siloed Inference **(b) Unified Reactive Scaling** **(c) Predictive Scaling**

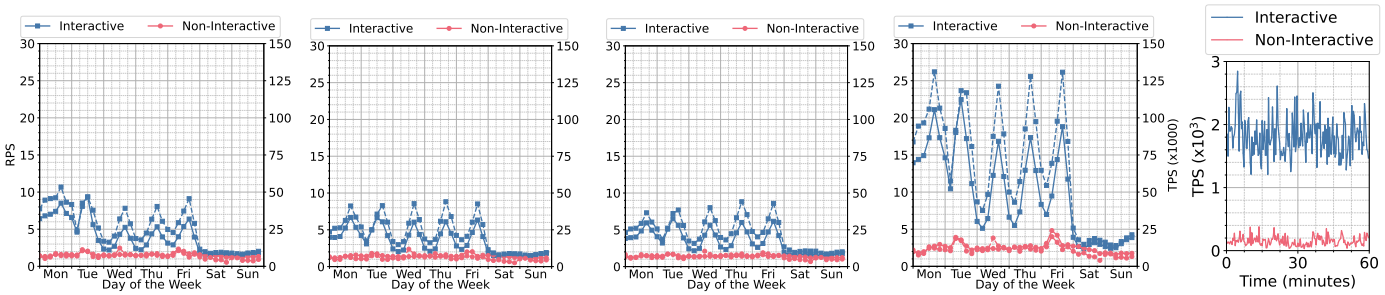
Figure 2: Routing and Scaling Strategies for LLM Inferencing

2 Related Work

Autoscaling for cloud computing. There is a long history of works on forecast-based autoscaling of CPU VMs [15, 33, 34, 39], which illustrates its need in all cloud services, challenges associated with operating at cloud scale, and the huge cost savings that can be obtained. For e.g. [15] says that even a 1% reduction in VM fragmentation can lead to $100M of cost savings in a year and we expect even higher savings for GPUs due to their higher cost [30]. Our work differs from these as we consider autoscaling of LLMs on GPUs which faces unique challenges due to the high latency sensitivity of interactive workloads, the significant SLA variation across workloads, and the high cost and latency of migrating and loading LLMs (hundreds of GB of data) on GPUs.

Efficient LLM serving. Several recent works have focused on optimizing the latency or throughput of LLM requests at a single model instance. There range from efficient attention computation [9, 19] and batched inference [4, 41] to user Quality-of-Experience (QoE) optimization [20] and providing fair service to all users [36]. However, these works typically only consider a single LLM instance and do not take into account multiple model types and GPUs.

LLM autoscaling. Since the advent of commercial LLM serving platforms [1–3], there has been some research into autoscaling LLM resources to meet workload requirments. [13, 18, 22] explored optimal placement of models on GPUs in a cluster by formulating different optimization problems. However, these optimizations are static and do not incroporate workload forecasts due to which they may not be effective in handling the high fluctuations in workload traffic observed in commercial LLM serving platforms (Fig. 3). There are also works that address specific challenges with LLM request serving such as handling pre-emption [23] and addressing startup and migration delays due to large model sizes [11]. However, these works do not consider the diverse workload mix and variations in model resource requirements that Pittacus does. Certain works do consider heterogeneous workloads, but focus on reactive approaches like fine-grained context-switching in Llumnix[38] to optimize for GPU memory defragmentation and load balancing and



(a) West US region (b) Central US region (c) East US region (d) Cumulative load (e) 1 hour load

Figure 3: RPS *(solid line)* and Total Input+Output TPS *(dashed line)* of IW and NIW requests summed across 4 LLM models for 1 week in Nov, 2024 for three regions (a)–(c) and their sum (d). The variations for a sample hour for the cumulative is in (e).

pre-empting NIW requests in ConServe[32] to prioritize interactive requests within a single instance. A recent work Chiron [31] performs backpressure based autoscaling to optimize for latency metrics for interactive requests across a cluster of instances, considering heterogeneous workloads. However, it assumes unlimited resources, which is unrealistic in real LLM platforms, and thus does not optimize for cost like Pittacus. Finally [16, 17, 26, 40] consider routing of LLM requests across different model instances for latency minimization or throughput maximization. Request routing is a critical component of LLM serving for short-term traffic fluctuations, it needs to be combined with dynamic model scaling as in Pittacus to handle longer-term traffic variations and avoid stale model assignments.

3 System and Application Model

We describe the cloud system, LLM deployment models and workloads, motivated by global installations at Cloud Provider X.

3.1 Cloud VMs and Model Instances

The CSP's system model comprises of multiple data centers (*regions*) connected with high bandwidth network, with≈ 50*ms* inter-region latency. Regions are assumed to be in USA (e.g., US-West, US-Central, etc) to avoid issues of data sovereignty. Each region has thousands of GPU VMs (e.g., Azure ND or AWS P4/P5 instances) with exclusive access to modern GPUS like NVIDIA A100/H100 or AMD MI300X and all the underlying server resources, capable of hosting LLM instances within regional capacity limits.

There are several standard LLM *model types* that are available, e.g., Llama 2, Bloom, GPT 3.5 turbo, etc. with associated default weights or custom weights based on fine-tuning. A *model instance* is one copy of a *model type* that can serve requests. Each instance may require multiple GPUs depending on the size of the LLM, e.g., GPT3 may require 9 H100s while Llama-3 needs 4 H100s [22]. Each VM is exclusively used by one LLM instance. There can be multiple instances for the same LLM type in a region. A *model endpoint* for that region receives requests to a model type and routes it to one of the instances based on round robin. There can be constraints on the minimum and maximum instance counts per endpoint, for robustness and to avoid one model dominating.

Tokens per Second (TPS) forms the key throughput capacity metric, and we focus on input TPS that a model instance can serve. The VM type and model type will determine the instance's performance, defined in terms of input TPS of throughput achieved at a target latency [22]. E.g. Llama2-70B (*Llama2*) [35] and Bloom-176B (*Bloom*) [5] models are known to achieve Q1–Q3 performance of 68–293 TPS and 50–177 TPS on 8× Nvidia A100 GPUs, and this improves to 95–522 TPS and 82–397 TPS with 8× H100 GPUs.

3.2 LLM Workloads and SLAs

The CSP supports multiple LLM inference workloads. *Interactive Workloads **(IW)*** with *low latency constraints (O(seconds))* from client-facting applications like chatbots, code generation, and email suggestions, require "fast" serving. In contrast *Non-Interactive Workloads **(NIW)*** such as nightly document summarization on enterprise repositories or detailed content generation, have *relaxed deadlines (O(hours))* and thus can tolerate 'slow" (or "no") serving.

For IW, clients for each product/service may use one or more pre-defined LLM types with thousands of input/output tokens per request. Traffic exhibits a diurnal pattern with upto tens of thousands of daily clients. E.g., Fig. 3 shows requests to all LLMs in three US regions, during one week in Nov, 2025, with Fig. 3e showing the granular request rate variability for one hour. For simplicity, we assume all clients are US-based since the regions are in the US. NIW also uses a set of pre-defined LLMs whose architectures often overlap with IW but have a lower and non-periodic request rate which is stable through the week.

These workloads have *different SLAs* defined. *Time to First Token (TTFT)* is the time between receiving a prompt to emitting the first response token and indicates responsiveness. In contrast, *End-to-End (E2E) time* is the time taken for generating all output tokens for a request, and influences both latency and throughput. IW primarily have a TTFT SLA, typically ranging from 1 second to 1 minute at 95%ile but can also have an E2E SLA. The SLA for NIW is typically a *deadline* for *batch completion* (e.g. 24*h* for summarizing a document repository) with less emphasis on per-request latency. We assume that serving each IW request within the latency SLA accrues a *utility* for the CSP, and serving an NIW request before its deadline accrues a *(lower) utility*.

3.3 Request Routing and Scaling

Routing Mechanisms. All IW requests are routed through a common LLM API service [27] to one of several LLM endpoints that can serve them (2b). This *global routing* to one of a configured set of regions can be based on network latency, geographical proximity or the current load on the region's endpoints. Then, a *region router* sends requests to endpoints for that model in that region, and further to instances within the selected deployment in a round robin manner to balance the load and address token skews. We assume a managed network and trusted security environment. There are no other security constraints that limit the mapping of instances to VM or requests to endpoints.

Scaling Delays. Creating a new LLM instance on VMs when scaling up a model type in a region has several *provisioning costs* that can vary based on the conditions. Allocating VMs to the instance is an initial cost. Then, if these VMs do not have the LLM already deployed on them, the model architecture and weights need to be copied and instantiated on them. The time taken depends on the model size, and on whether they are already available in a local repository in that region, e.g., taking $\approx 10mins$, or need to be moved from a remote region, e.g., taking $\approx 2h$. If the VMs already have the LLM architecture deployed from a prior provisioning but with different weights, only the weights need to be updated and the cost reduces. When an instance is being provisioned the VMs are not available for use. So the model provisioning time constitutes *wasted GPU cycles*. In addition, there are latency costs to acquiring a GPU VM and updating all upstream services such as load balancer, etc. Given that this total time can run from mins–hours, frequent re-provisioning is inefficient.

Spot Instances. The workloads are executed on LLM instances that are provisioned in a private network space. However, if the endpoints of common LLM model types are idle, they can be leased to external users as (preemptible) *spot LLM instances* for inferencing at a lower cost, and reclaimed when the internal demand increases. Switching an instance from private to spot, and the reverse, is relatively fast, ≈ 1 mins. Typically, the *utility benefits* of leasing out spot instances is lower than that gained from executing the internal IW and NIW workloads. But this is still better than keeping the VMs idle. During some periods, 25% of instances in a region may be donated to spot; *this is a lost opportunity cost we aim to fix.*

4 Motivation: Effective Resource Scaling

We study the effect of scaling resources allocated to an LLM type on capital efficiency and SLA compliance for both IW and NIW workloads. Since GPU VMs are costly, the goal is to maximize their utility – either by serving IW or NIW workloads, or leasing the idle capacity as spot instances.

The baseline *siloed deployment* is currently used at Cloud Provider X for serving LLM inference requests (Fig. 2a). It maintains separate instance pools per workload (IW/NIW) and LLM type in each region. Here, separate pools of LLM instances are maintained for IW and NIW workloads, for each LLM type in a region. It applies a greedy scaling policy, adding instances to the region endpoing when effective memory utilization for the instances increases above 70%, and returns an instance to spot if the utilization drops below 30%. The effective memory utilization excludes memory used for model weights and is a reliable proxy for the request load. These scaling decisions are made per request, constrained by instance limits per model type. However, siloing fragments VMs into captive pools, often leading to suboptimal utilization.

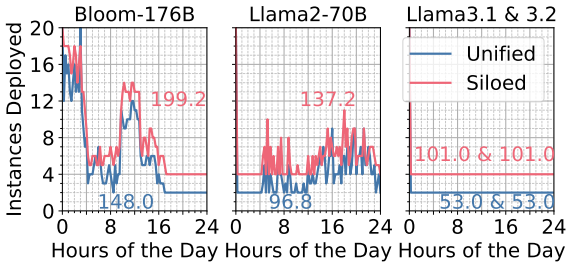


Figure 4: Model instance count across time and total instance-hours for unified vs. siloed strategies for Tuesday workload in US-West, with peak 20 instances per model.

As an alternative, we propose a *reactive scaling heuristic* using a *unified instance pool* to serve both IW and NIW across all models in a region (Fig. 2b), allowing dynamic sharing of spare capacity across workload types [31]. NIW requests are queued and served only when instance utilization drops below a limit (50–60%) or when their SLA deadline is approaching, ensuring that they leverage spare capacity without affecting IW performance or triggering scaling-up. This enables shared use of model instances across workloads and allows flexible deployment of any LLM type, improving overall VM utilization over donating to spot. As mentioned, switching an LLM instance between spot and internal endpoints takes 1 min, while changing a VM's hosted instance requires ≈ 10 mins. Scaling events are triggered based on effective utilization seen at regional endpoints, with a 15-second cooldown enforced between events.

We evaluate the *siloed* and *unified reactive* scaling heuristics for four LLMs (Bloom, Llama 2, Llama 3.1, Llama 3.2) deployed in three U.S. regions (East, West, Central). Each region has 20 instances per model: 16 for IW and 4 for NIW in the siloed approach, and all 20 in a unified pool with the unified heuristic. The minimum and maximum instance counts per endpoint are 2 and 3, respectively, which is the current practice at Cloud Provider X ensuring fault tolerance and load balancing. We use a realistic simulation harness for Pittacus which is built on top of Splitwise [29], whose results closely match real-world behavior (see Section 7.1). We use 8xA100s per instance, and replay one day of workload data (Tuesday) from the week shown in Fig. 3, which has 1.4M IW and 0.2M NIW requests.

Fig. 4 illustrates the instance count at US-West every 15-mins for the siloed and unified approaches for each model for that day (Fig. 3a is the input workload). Our unified deployment strategy consumes fewer model instance hours (area under the curve) for all four models despite using similar scaling thresholds for both deployments. This is because the instances in the unified deployment are not tied to a workload type (IW or NIW) and can be used dynamically based on demand. In case of Llama 3.1 and 3.2, since they received few requests and maintained the minimum instance count – siloed allocates 2 instances each for IW and NIW while unified shares the 2 instances for the IW and NIW requests (the Llama models behave identically and are are clubbed into one plot).

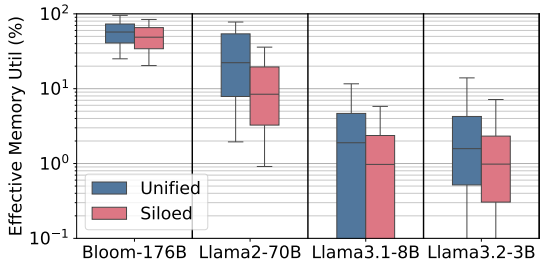


Figure 5: Memory usage by strategies, for Tuesday workload at US-West. The unified approach uses 34.5% fewer instance-hours than the siloed approach across all four models.

Table 1: 95%ile of TTFT and end-to-end latencies for serving different models using siloed and unified approach.

Metric	Bloom-176B		Llama2-70B		Llama3.1-8B		Llama3.2-3B	
	Siloed	*Unified*	*Siloed*	*Unified*	*Siloed*	*Unified*	*Siloed*	*Unified*
TTFT (s)	14.5	12.9	34.9	34.5	1.0	1.0	1.0	1.0
E2E (s)	55.3	53.3	98.3	99.1	10.6	10.5	19.2	18.9

This consolidation is reflected in the higher memory utilization for the unified heuristic in Fig. 5, while not sacrificing the SLA latency for IW. In both approaches, the change in the 95%ile TTFT ranges from 0 to 12% (see Table 1). Both approaches are able to process all the queries in the trace, but unified is able to use fewer resources, and donate an additional 52 instance-hours to spot than the siloed approach.

Also, the memory utilization of Llama 2 is generally less than Bloom, indicating that using the unified pool to allocate across model instances (inter-model scaling) can further help adapt better to complementary demand compared to siloed that does not allow allocation of VMs across models.

Summary. From these experiments, we see that mixing IW and NIW workloads through a unified pool of instances improves resource utilization and cost efficiency, opening new optimization avenues through flexible NIW processing (c.f. Table 1 and Fig. 4). However, reactive scaling can harm IW SLOs due to under-allocation or raise the costs due to over-allocation (Fig. 1). Hence, this motivates the need for proactive scaling that leverages TPS predictions for the workloads (Fig. 3) coupled with the unified pool across workloads.

In the following sections, we first formulate this as an optimization problem and propose an Integer Linear Programming (ILP) to find the optimal allocation of GPUs (§ 5.1), describe the overall architecture of our resource management systems (§ 6) and finally, we show the effectiveness of our system(§ 7).

5 Optimization Problem

We solve the problem of request routing and capacity allocation to serve fast and slow LLM inferencing workloads within the required SLA while maximizing utilization.

Definition. *Given a captive set of VMs of specific types in multiple regions,*

Table 2: Variables used in optimization problem

Symbol	Type	Description
l	`int`	Model types
r	`int`	Number of regions
$n_{i,j}$	$[\texttt{int}]_{l\times r}$	Instances of model i at region j
$\rho_{i,j}(w)$	$[\texttt{int}]_{l\times r}$	TPS requested for model i from clients at region j for future time window w
θ_i	$[\texttt{float}]_l$	TPS provided by model i
α	`float`	Cost of acquiring GPU VM
σ_i	$[\texttt{float}]_l$	Cost of starting a model i
$\delta_{i,j}$	$[\texttt{int}]_{l\times r}$	ILP output: optimal number of changes in VM allocation

- *we need to continuously ensure that the right number of model instances of different model types are provisioned as endpoints at the regions, and*
- *route the incoming workload across these endpoints,*

such that

- *we maximize the utility of the workload requests completed within their SLA, and*
- *maximize the capacity utilization of the VMs for the interactive workloads.*

We have two parts to solving this high level problem.

First, we need to **optimally provision instances** for model endpoints in different regions to handle this workload, within the available VM capacities. Since there is an overhead for (re)provisioning an LLM instance set onto VMs, we need to consider the inefficiency due to this process when the GPU VMs are not serving. Given this, such reprovisioning decisions are only viable at a coarse granularity, e.g., every 15 mins – while some reprovisioning such as reclaiming spot instances can be fast, e.g., done each 1 min.

Second, we need to **route the requests** to the model instances in different regions while meeting the SLA. These routing decisions can use real-time information on the load and responsiveness of the region endpoints. As part of reprovisioning resources and routing, we need to use relevant tools like the SplitWise simulator [29] to estimate the expected latency for serving requests at a certain request rate to ensure we meet the SLA. Any spare capacity of model instances that are not being used for workloads can be released to external spot instances.

Next, we formulate these as an optimization problem.

5.1 Problem Formulation

Table 2 has the notations used in the problem definition.

Constraints. Say, the current number of VMs assigned to a model i at region j at a given time is $n_{i,j}$. Let $\delta_{i,j}$ be the optimal number of changes to be made to this VM count to service all IW requests in the next hour.

Servicing Interactive Workload within each Region. Say the forecasted rate of tokens to be served per second for an IW workload for model i at region j during each time window w over the next decision making window of, say, 1 hour, is $\rho_{i,j}(w)$. Each model type in a region should be able to serve at least a fraction $0 < \epsilon \leq 1$ of it's peak future request load received from clients within its region in real-time. Excess load $(1-\epsilon)$ can be rerouted to other regions to

reduce the number of model-instance changes needed.

$$(n_{i,j} + \delta_{i,j}) \times \theta_i \geq \max_w \ \epsilon \times \rho_{i,j}(w) \ \forall i, j$$

Servicing Interactive Workloads across all Regions. Next, we ensure that all IW requests for each model i received from across all regions j can cumulatively be processed using its model instances present across all region, with rerouting.

$$\sum_j (n_{i,j} + \delta_{i,j}) \times \theta_i \geq \max_w \sum_j \rho_{i,j}(w) \ \forall i$$

Non-negative number of instances. We should never deallocate more models than are allocated, $\delta_{i,j} \geq -n_{i,j}$.

Objective. We minimize the wasted resource overheads when provisioning VMs and instances required to support IW workloads while maintaining their SLAs. We incur two overheads when provisioning a new model instance: (i) VM start up cost to instantiate a new VM (γ), and (ii) the deployment cost (μ) when loading the model and its weights on a VM, when we have acquired and are paying for the VM but are unable to use it yet to serve requests. This makes acquiring new VMs less attractive than spot instances.

$$\gamma = \left(\alpha \times \sum_{i,j} \delta_{i,j}\right)$$

$$\mu = \sum_i \sum_j \left(\sigma_i \times \max(0, \delta_{i,j})\right)$$

$$\textit{The objective is} \quad \arg\min(\gamma + \mu)$$

6 Architecture of Pittacus

We next describe the approach and design of our proposed Pittacus framework solves the proposed problem.

6.1 Overview

Fig. 2c shows the Pittacus LLM serving architecture operating across regions. At the frontend, Pittacus provides serving APIs similar to other LLM serving platforms. For IW requests, it uses a real-time streaming API that returns outputs once each token is generated [24, 28]. For NIW requests, it adopts an interface similar to Batch APIs [25, 27] which takes requests and returns responses asynchronously. The global router receives a request and routes it to the relevant region hosting the LLM instances which can service the request (§ 6.2). Other components orchestrate the forecasting and optimization logic (§ 6.4), scaling logic (§ 6.5 and NIW queue manager (§ 6.3). These are discussed next.

6.2 Routing Logic

The routing module includes the routing to regions by the global router, routing to endpoints within a region, and routing locally among the instances of an endpoint.

Global routing for IW requests. When IW requests are received at the global router, the region routing logic checks the effective memory utilization of all the available regions hosting the model and routes the request to the region with memory utilization less than the desired threshold (70%). The threshhold is chosen in accordance to production systems at Cloud Provider X. If multiple regions match, we can specify an order of preference, e.g., based on network proximity. The *effective memory utilization* is calculated as the ratio of the *sum of the effective memory utilized* to the *effective memory available* across all instances for a model in a region. The effective available memory for a model instance is obtained by subtracting model weights from the total VM memory. If none of the preferred regions has memory utilization less than the threshold, then the region with the least memory utilization among the choices is selected.

Global routing for NIW requests. NIW requests are sent by the global router to a *Queue manager* that holds these requests (Fig. 2c). Each model endpoint at the regions periodically send a signal to the Queue manager when their effective utilization falls below a specific threshold (60%, in our experiments). Then, the NIW requests waiting in the queue for that region and model are incrementally removed by the Queue Manager and routed to that available endpoint, as described in § 6.3.

Routing logic at region endpoint. For both IW and NIW requests, we route requests arriving at a region to the least loaded deployment endpoint for that model, based on the effective memory utilized. Requests are sent to the instance with the minimum remaining tokens to proces, based on the "Join The Shortest Queue Logic" [14].

Scheduler at the model instance. A local queue is maintained at the instance as well. IW requests arriving at an instance are assigned priority 0 and available immediately for inference. The scheduler at the model instance selects requests and batches them for inference in a first-come, first-served manner. NIW requests may also arrive with a priority 0, as set by the Queue Manager when their deadline is approaching fast, and are considered on par with IW requests. NIW requests with the default priority 1 are selected for inference if there are no priority 0 requests waiting at the instance queue.

6.3 Queue Manager for NIW

The Queue Manager take care of asynchrnously routing NIW requests to a specific region and endpoint. Upon receiving a signal from an model endpoint on its capacity availability, an asynchronous feedback logic initiates several steps. It pulls the requests that are queued at the Queue Manager for that model type and routes them to the region from which the signal was received. All NIW requests whose age is less than 10 hours are given a priority of 1, while requests that have an age greater than 10 hours are given a priority 0, similar to IW requests. If the signal received indicates that the effective memory at the regional endpoint is less than 60%, one request is removed from the Queue Manager added sent to the endpoint; if it is less than 50%, two requests are sent.

6.4 Forecast Logic and Optimization Module

For IW requests, we forecast the rate of input token requests that will be received per region per model type using an ARIMA model [37]. We find this be accurate enough to predict the diurnal load for each model from a region. From the forecasting module, we take the maximum TPS expected in the next hour to estimate the TPS capacity required for an IW model. We add a buffer of β to this forecasted TPS to handle transient bursty workloads and to offer spare capacity to serve NIW requests. We calculate the buffer as $\beta = 10\%$ of the NIW load we received in the past hour. The output of the forecast module is passed to the optimization module, which uses an ILP solver to solve the optimization problem in § 5.1. This returns $\delta_{i,j}$,

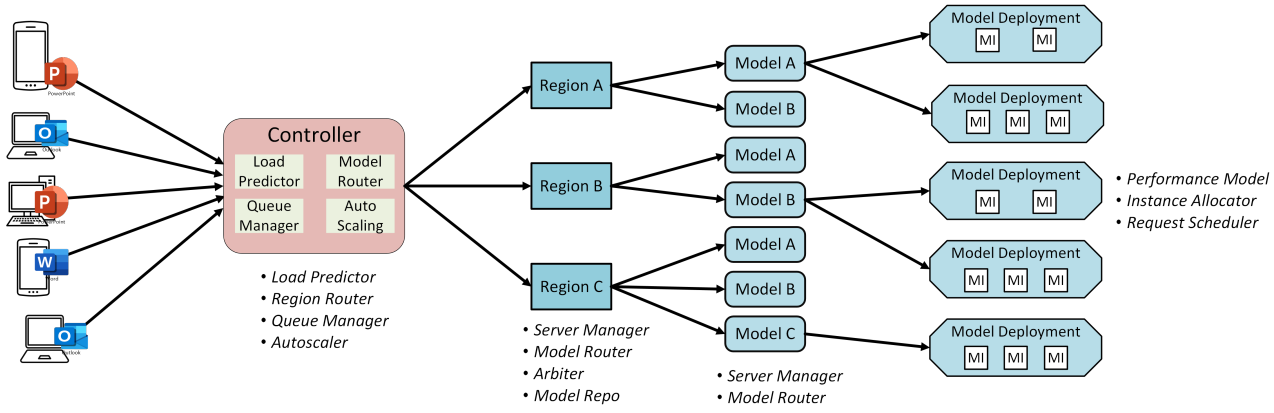


Figure 6: Overview of the Pittacus simulation setup used for evaluation, corresponding to the architecture in Fig. 2c.

which is the change in the number of instances assigned to model i at region j. The forecast and optimization module is invoked every hour.

6.5 Scaling Logic

The long-term scaling logic uses the hourly output from the forecasting module, which solves the optimization problem using ILP. If the recommendation, $\delta_{i,j}$, for a specific region and model type is positive, we need to scale out the instances; and if the prediction is negative, we need to scale in. For scale out, we first reclaim spot instances of the identical model type which are faster to acquire, and if none are available, we reclaim spot instances from other model types which can be slower to provision. Similarly, for scaling down, we donate the instances to the spot instances of the same model type.

We have a few choices on when to initiate the scaling.

Immediate (LT-I). One naive approach is to immediately scale instantly to the instance count recommended by the long-term scaler every hour. We term this as Immediate (LT-I). However, the recommendation is based on the peak load that will occur in the next 1 hour. So scaling out immediately can cause transient over-provisioning, well before the peak load actually arrives.

We introduce two additional systematic scaling strategies to pace the rate at which the deployment state catches up with the forecasted load. These approaches help improve the utilization of VMs and utility serving requests, making spare capacity available to other models that may require it in the same region.

Instance Utilization (LT-U). If the scaler suggests scaling out the instances, we do so only when the effective memory utilization actually starts increasing and goes over the threshold of 70% used in out experiments. We keep increasing the instance count as long as this threshold is breached and until we achieve the instances count suggested by the optimization model. Similarly, if the scaling logic suggests downscaling, we do so when the utilization goes below the 30% threshold, again until we have reduced to the suggested number of instances. The region endpoint provides the effective memory usage every time it receives a new request.

Instance utilization and ARIMA gap (LT-UA). Our optimization model can make erroneous recommendations if the ARIMA predictions are inaccurate, which is bound happen if bursty requests occur often. As before for LT-U, we defer the scale out or in based on the memory usage thresholds actually being breached in either direction. However, we do not strictly stop the scale out or in if the instance count reaches the target count. Instead, during the last 20 mins of the hour, if we have reached the scale out instance count and the observed TPS load is $\geq 5\times$ of what ARIMA predicted, then we continue to scale up the instance count. On the other hand, if the TPS load received is $\leq 0.5\times$ of what the ARIMA prediction, we continue scaling down. Here, we switch from a memory-based strategy to a traffic-based strategy during the last 20 mins of the decision window if the memory-based strategy has reached its goal.

7 Evaluation

Our detailed evaluation aims to answer the following questions:

(1) *How effectively can the proposed approach (Pittacus) utilize GPU resources while maintaining latency targets for both interactive and non-interactive workloads?* (§ 7.3.1, §7.3.2)
(2) How does the proposed approach (Pittacus) compare to SOTA scaling strategies? What are the cost advantages of using our system compared to existing methods? (§7.3.3, §7.3.4)
(3) *Can Pittacus respond quickly and maintain latency targets for both IW and NIW in the face of unexpected load bursts and prediction errors?* (§7.3.5)
(4) *How does Pittacus scale over a week-long period, taking into account diurnal patterns and differences between weekday and weekend workloads?* (§ 7.3.6)

7.1 Implementation and Simulating Setup

Experimenting with different scaling mechanisms and verifying their effectiveness with multiple LLM types and GPU VMs can prove quite costly. To alleviate this pressure and enable easier research in this direction, we extend existing the SOTA LLM simulator, *Splitwise* [29], to simulate a datacenter running multiple models on a variety of hardware. The Splitwise simulator models the various components in the LLM Serving stack in a Python based discrete

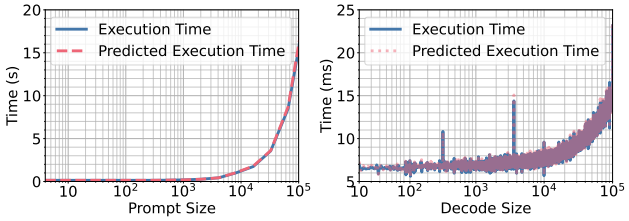


Figure 7: Comparison of batch execution time predicted by Splitwise vs Real model instance for prompt phase (left) and decode phase (right).

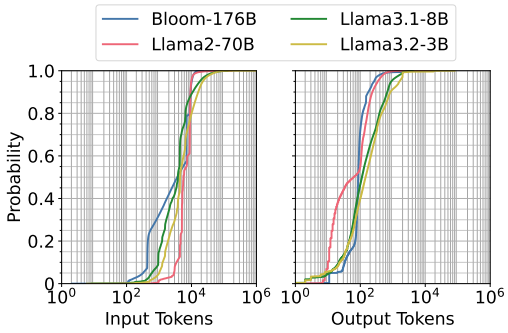


Figure 8: CDF of Prompt, Output and Total Token Counts in log scale. Majority of requests in real world traces have greater than 1000 prompt tokens and fewer than 1000 decode tokens. Token distribution varies per model.

event simulation setup. This includes the request queue, router and scheduler along with the instances. To predict request runtimes, i.e., TTFT and E2E latencies, it uses a robust interpolation-based performance model that uses real inference traces. A single instance of Splitwise is equivalent to one model instance deployment in a specific region.

We begin by independently verifying the accuracy of the Splitwise simulator (Fig. 7), which reports <5% MAE on recent LLM models and hardware for both the prompt and decode phases. Since both these phases are within a reasonable error margin, the simulator will be accurate in predicting the TTFT and overall E2E latency of processing our request traces as well. Using this simulator instance as the atomic entity whose accuracy we have verified, we further build our evaluation harness to launch multiple instances of these simulators and create a unified event queue for them, taking into account routing and iterations through the model. The different components of our simulator are shown in Fig. 6.

Our simulator mimics multiple regions across the globe and is implemented at a granular level in order to make it flexible for different settings. Thus, it can be used for experimenting and evaluating new routing and batching algorithms as well, in order to study the impact of these changes across the entire inference serving stack. We will open source this work to give service providers holistic simulations and enable work on full stack optimizations.

We first profile Bloom-176B, Llama3.1-8B, Llama3.2-3B, and Llama2-70B on a VM with A100-80GB with various input/output sizes as given in [29]. The simulator provides TTFT, Time Between Tokens (TBT), E2E per request, and machine level performance using the performance models trained on characterization profiles. The accuracy of the performance model is validated using mean absolute percentage error (MAPE) on a 80:20 train:test dataset split. The error is less than 3%, as shown in Fig. 7.

Pittacus's ARIMA-based load predictor, region router, queue manager and autoscaler are embedded within the controller module of the simulator. The controller uses these components to coordinate with different regions. Each region has a server manager, arbiter, model repository and model router. The model repository stores all the model weights for ease of deploying a model. Furthermore, each model endpoint in a region has a server manager and model router. The request scheduler, instance allocation, and performance models to capture request-level metrics and machine-level utilization are present at each model deployment. We run our experiment harness for four models and three regions. The minimum instance count within a deployment is assumed to be two, and the maximum instance count in a model deployment is assumed to be three.

GPUs and LLM Models. We evaluate workloads for three regions, US East, US West and US Central, and four standard LLM model types, Bloom, Llama-2, Llama-3.1 and Llama-3.2, used by IW and NIW with their default weights. All the model types are assigned 20 instances per region. We assume homogeneous hardware in the simulation and also the number of GPU cards needed by each models as identical. The *performance (TPS)* for each LLM type instance on each GPU type is shown in Fig. 7.

Deployment Characteristics. There are network overheads between different regions that affect the latency for client requests from one region being routed to instances in different regions. The realistic latency distributions from source to sink regions is collected and used by the simulator from Cloud Provider X. For around 90% of the regions, the network latency is within 500ms with less than 2% of cases having a network latency of 2.5s.

There is a latency for deploying a new LLM model onto a set of GPU VMs. These times depend on the LLM model size. We assume the redeployment time for a model with weights available in the same region as around 10 minutes for all the models regardless of their parameter size. Though the time varies slightly based on the time it takes to transfer weights, the redeployment time of a model for which weights are absent in the local region is ≈2 hours. These assumptions are consistent with our observations from Cloud Provider X.

In case existing spot instances are being reclaimed, the time to do so is a median of 1 minute and a maximum of 5 minutes. We assume that the time for routing the client request to the target region's endpoint is negligible. The simulator also captures the unavailability of the VMs when being re-provisioned.

Workload Characteristics. We consider 3 client regions (US-E(all), US-C(all), US-W(all)) and focus on a one week period in November, 2024 for workloads generated for the 4 major model types, as shown in Fig. 3. This forms the core dataset for our IW and NIW workloads as described earlier. For each of these LLM types, Fig. 8 provides a distribution of the input and output token counts. In general, the majority of requests have an input token count greater than 1k, while the majority of the output token count distribution is before 1k and varies with model type.

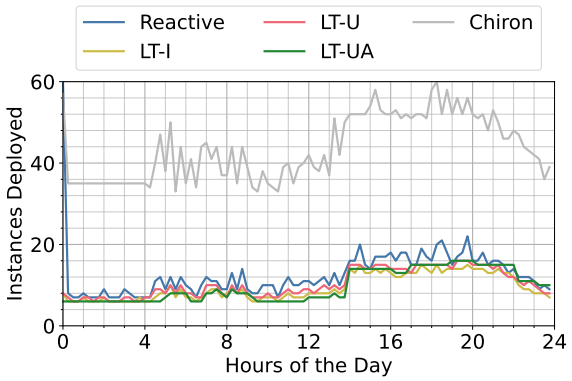


Figure 9: Aggregated sum of instances deployed across regions for Llama-2 on a peak traffic day. Area under curve for Reactive, LT-I, LT-U, LT-UA and Chiron are 302.25, 227.25, 247.5, 233.25, 1046.25 , instance-hours, respectively. This translates to savings of roughly $0.5M when deploying the system on our Cloud Provider X over a week ($98.32 per hour for H100 clusters at the time of writing)!

7.2 Baselines

In the baselines, we consider the unified reactive heuristic from § 4 that represents the status quo. Whenever a request arrives at the regional endpoint of a model type, the effective utilization is measured, and if the value is greater than 70% (< 30%), the instance is scaled out (in). This approach is followed every time a request reaches an endpoint, with a cooldown period of 15s between scaling events. Furthermore, scaling events always happen using spot instances – surplus instances are donated to spot, and instances are acquired from spot when needed. Inter-model scaling is also allowed using spot instances, i.e., redeployment of a new model to replace the spot instance hosting another model type. For all the experiments, we discuss four strategies: unified reactive heuristic (reactive), LT-I, LT-U, and LT-UA.

For state of the art, we implement Chiron[31] on our simulator for evaluation against Pittacus. We initialize the system with ten interactive, five mixed and five batch instances of each model type in every region. We set Θ to 0.6 in Chiron's interactive autoscaling algorithm for ideal performance on our traces as guided by their work. We keep the rest of the components like global routers and schedulers consistent between Chiron and Pittacus for a fair comparison.

7.3 Results

7.3.1 Effectiveness of Proactive Strategies in Reducing GPU-hours. We first evaluate the effectiveness of Pittacus for a single day trace. Figure 9 shows the trends of instance hour usage by hour, aggregated across all the three regions for Llama-2 70B model. Our forecast aware strategies consistently use less instances as compared to the reactive strategy, with LT-I, LT-U and LT-UA using 24.8%, 18.2% and 22.8% less instance hours respectively. This is intuitive as LT approaches do not react to momentary bursts in traffic and scale based on the forecasts. These results are also evident in Fig. 10, which shows LT strategies are better for all regions.

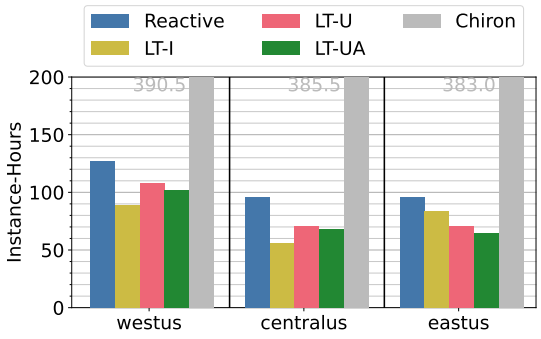


Figure 10: Llama-2, Instance-Hours for different strategies. Pittacus is able to reduce the instance hours across different regions with different workload patterns.

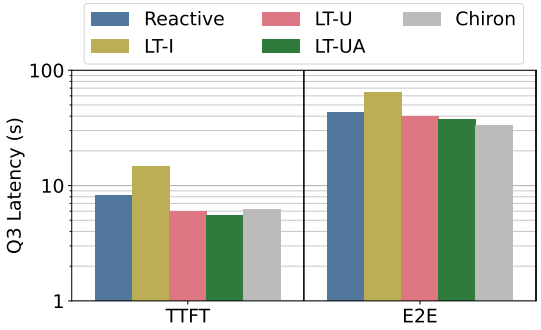


Figure 11: Latency Metrics across regions for Llama-2 on a peak traffic day. TTFT and E2E latency of serving requests is not harmed by proactive scaling approaches with real world workloads containing bursts.

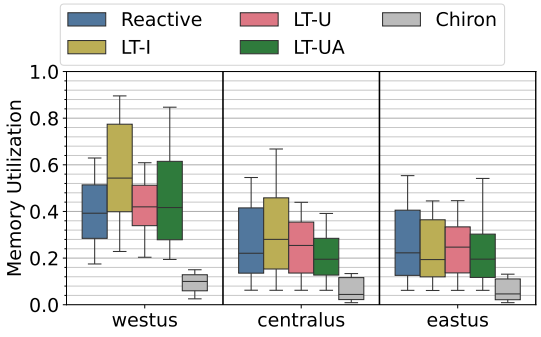


Figure 12: Llama-2, mean memory utilization for different strategies. Memory utilization in LT-I increases as immediate de-allocation of instances can lead to higher pressure on the model instances. This problem is resolved in LT-U and LT-UA with heuristic based short term scaling.

7.3.2 GPU Cost Reduction without SLO Violations. LT-I utilizes less GPU hours but it slightly harms the TTFT and E2E latency of requests, as evident from Figure 11. This is because while immediately scaling up does not significantly benefit requests when traffic is lower, immediate scale down slows down requests, potentially harming their SLOs. These issues are fixed in LT-U and LT-UA, where instances are scaled on demand. These strategies

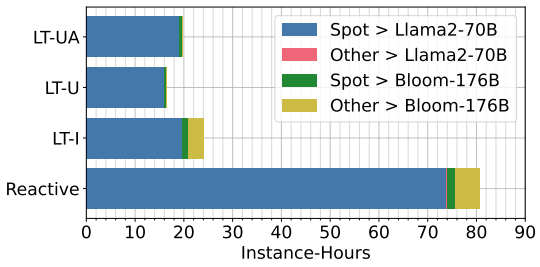


Figure 13: Overhead of scaling for different strategies. 'Spot>Llama' is time taken to acquire spot by Llama. 'Other>Bloom' is latency to acquire spot instance of other model and redeploying Bloom. Pittacus wastes less GPU cycles by avoiding frequent scaling up of model instances.

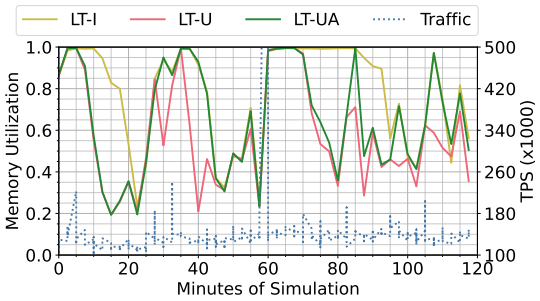


Figure 14: The performance of LT-UA when sudden synthetic request bursts are introduced. The effective memory utilization of Llama-2 in US-East remains low for LT-UA, while the value spikes for other strategies. We introduced two bursts in the incoming traffic, which lead to an initial spike in the memory utilization of all three strategies. LT-UA was able to recover quickly from this by allocating instances above the threshold set up the forecasting logic, while LT-I and LT-U's utilization remained high.

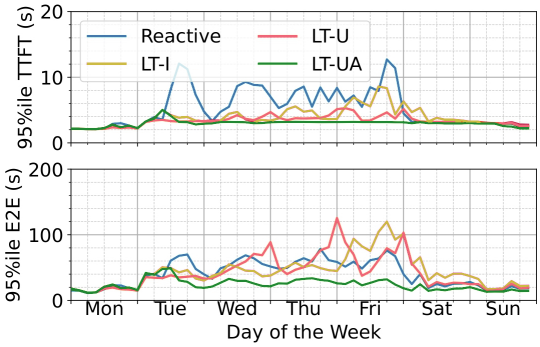


Figure 15: 95%ile latency metrics binned by 3 hours for Llama2-70B. Pittacus reduces the 95% TTFT and E2E latency of request as compared to reactive scaling as it allocates instances proactively when incoming traffic is increasing.

helps us downscale only when we can do so while serving low latency requests, while making sure we do not up (or down) scale too much.

7.3.3 Comparison with Chiron. Optimizing solely for SLA as done in Chiron can lead to increased instance demand without significant tail latency improvements, as evident in Figures 9 and 11. Additionally, Chiron exhibits lower hardware utilization compared to our approach and even reactive scaling mechanisms. This inefficiency stems from Chiron's reliance on offline profiles for infrastructure scaling rather than incorporating online memory utilization metrics.

7.3.4 Scaling Costs. We see in Figure 13 that Pittacus is able to reduce the GPU cycles wasted during instance deployment by about 70%. Due to fluctuations in traffic, reactive scaling approaches generally waste GPU cycles in constant scaling up operations. With the forecast knowledge, Pittacus's methods are able to reduce the number of times we upscale as well, resulting in much better utilization of acquired hardware.

7.3.5 Burst Management using Pittacus. As shown in the blue curve of Fig. 14, we randomly increase the incoming load to 8x in order to simulate sudden bursts in traffic. While LT-U and LT-I are able to maintain their latency and memory utilization when small bursts in traffic happen, they do not scale above the threshold set by the ILP and the ARIMA forecast even when large bursts occur. This is evident in the peaking latency metrics during this period shown in Fig. 14, where we see that the green curve of LT-UA is able to reduce back it's memory utilization faster than LT-I and LT-U. Therefore, in such scenarios, LT-UA is able to cope with the uncertainity much better. As discussed in § 6.5, we set the threshold to scale up at 5x predicted traffic.

7.3.6 Validation on Week Long Trace. Fig. 15 displays the 95%ile of TTFT and E2E latency over the course of one week. The insights gained from a one-day trace also apply in this case. The reactive strategy shows inferior performance, while other strategies achieve better performance metrics. LT-U and LT-UA behave similarly during weekdays, with a slight change in performance at the start of the weekend. This indicates the effect of the LT-UA strategy across longer time scales, which accounts for errors in ARIMA forecasts when the trend in TPS differs during the weekend. Overall, Pittacus scales well with longer traces and for different request rates.

8 Conclusions

We present Pittacus, a holistic system for serving LLM inference requests with a wide range of SLAs, which maintains better GPU utilization, reduces resource fragmentation that occurs in silos, and increases utility by donating surplus instances to Spot instances. Pittacus achieves this through its unique elements, namely, a holistic deployment stack for requests of varying SLAs, its async feed module, and long-term aware proactive scaler logics that capitalize on the underutilized instances of another model in the same region by inter-model redeployment.

Future work includes extending Pittacus to accomodate workloads with a continuum of SLAs and conducting extensive studies on the benefits of the proposed approach with deployments across heterogeneous hardware types. We plan to open-source our trace data and simulator.

References

[1] [n. d.]. ChatGPT. http://chat.openai.com.

[2] [n. d.]. Copilot. http://copilot.microsoft.com.

[3] [n. d.]. Gemini. http://gemini.google.com.

[4] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming {Throughput-Latency} Tradeoff in {LLM} Inference with {Sarathi-Serve}. In 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24). 117–134.

[5] BigScience. [n. d.]. Introducing The World's Largest Open Multilingual Language Model: BLOOM [Online]. In https://bigscience.huggingface.co/blog/bloom.

[6] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, Erik Brynjolfsson, S. Buch, Dallas Card, Rodrigo Castellon, Niladri S. Chatterji, Annie S. Chen, Kathleen A. Creel, Jared Davis, Dora Demszky, Chris Donahue, Moussa Doumbouya, Esin Durmus, Stefano Ermon, John Etchemendy, Kawin Ethayarajh, Li Fei-Fei, Chelsea Finn, Trevor Gale, Lauren E. Gillespie, Karan Goel, Noah D. Goodman, Shelby Grossman, Neel Guha, Tatsunori Hashimoto, Peter Henderson, John Hewitt, Daniel E. Ho, Jenny Hong, Kyle Hsu, Jing Huang, Thomas F. Icard, Saahil Jain, Dan Jurafsky, Pratyusha Kalluri, Siddharth Karamcheti, Geoff Keeling, Fereshte Khani, O. Khattab, Pang Wei Koh, Mark S. Krass, Ranjay Krishna, Rohith Kuditipudi, Ananya Kumar, Faisal Ladhak, Mina Lee, Tony Lee, Jure Leskovec, Isabelle Levent, Xiang Lisa Li, Xuechen Li, Tengyu Ma, Ali Malik, Christopher D. Manning, Suvir P. Mirchandani, Eric Mitchell, Zanele Munyikwa, Suraj Nair, Avanika Narayan, Deepak Narayanan, Benjamin Newman, Allen Nie, Juan Carlos Niebles, Hamed Nilforoshan, J. F. Nyarko, Giray Ogut, Laurel Orr, Isabel Papadimitriou, Joon Sung Park, Chris Piech, Eva Portelance, Christopher Potts, Aditi Raghunathan, Robert Reich, Hongyu Ren, Frieda Rong, Yusuf H. Roohani, Camilo Ruiz, Jack Ryan, Christopher R'e, Dorsa Sadigh, Shiori Sagawa, Keshav Santhanam, Andy Shih, Krishna Parasuram Srinivasan, Alex Tamkin, Rohan Taori, Armin W. Thomas, Florian Tramèr, Rose E. Wang, William Wang, Bohan Wu, Jiajun Wu, Yuhuai Wu, Sang Michael Xie, Michihiro Yasunaga, Jiaxuan You, Matei A. Zaharia, Michael Zhang, Tianyi Zhang, Xikun Zhang, Yuhui Zhang, Lucia Zheng, Kaitlyn Zhou, and Percy Liang. 2021. On the Opportunities and Risks of Foundation Models. ArXiv (2021). https://crfm.stanford.edu/assets/report.pdf

[7] Sophia Chen. 2024. A day in the life of the world's fastest supercomputer. Nature 633 (2024), 22–25. https://doi.org/10.1038/d41586-024-02832-5 News Feature, Correction published 11 September 2024.

[8] Hao Cui, Zahra Shamsi, Gowoon Cheon, Xuejian Ma, Shutong Li, Maria Tikhanovskaya, Peter Norgaard, Nayantara Mudur, Martyna Plomecka, Paul Raccuglia, Yasaman Bahri, Victor V. Albert, Pranesh Srinivasan, Haining Pan, Philippe Faist, Brian Rohr, Michael J. Statt, Dan Morris, Drew Purves, Elise Kleeman, Ruth Alcantara, Matthew Abraham, Muqthar Mohammad, Ean Phing VanLee, Chenfei Jiang, Elizabeth Dorfman, Eun-Ah Kim, Michael P Brenner, Viren Jain, Sameera Ponda, and Subhashini Venugopalan. 2025. CURIE: Evaluating LLMs On Multitask Scientific Long Context Understanding and Reasoning. arXiv:2503.13517 [cs.CL] https://arxiv.org/abs/2503.13517

[9] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. Advances in Neural Information Processing Systems 35 (2022), 16344–16359.

[10] Rafael Ferreira da Silva, Rosa M. Badia, Deborah Bard, Ian T. Foster, Shantenu Jha, and Frédéric Suter. 2024. Frontiers in Scientific Workflows: Pervasive Integration With High-Performance Computing. Computer 57, 8 (2024), 36–44. https://doi.org/10.1109/MC.2024.3401542

[11] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. 2024. {ServerlessLLM}:{Low-Latency} Serverless Inference for Large Language Models. In 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24). 135–153.

[12] Luigi Fusco, Mikhail Khalilov, Marcin Chrapek, Giridhar Chukkapalli, Thomas Schulthess, and Torsten Hoefler. 2024. Understanding Data Movement in Tightly Coupled Heterogeneous Systems: A Case Study with the Grace Hopper Superchip. arXiv:2408.11556 [cs.DC] https://arxiv.org/abs/2408.11556

[13] Tyler Griggs, Xiaoxuan Liu, Jiaxiang Yu, Doyoung Kim, Wei-Lin Chiang, Alvin Cheung, and Ion Stoica. 2024. M\'elange: Cost Efficient Large Language Model Serving by Exploiting GPU Heterogeneity. arXiv preprint arXiv:2404.14527 (2024).

[14] Varun Gupta, Mor Harchol Balter, Karl Sigman, and Ward Whitt. 2007. Analysis of join-the-shortest-queue routing for web server farms. Performance Evaluation 64, 9-12 (2007), 1062–1081.

[15] Ori Hadary, Luke Marshall, Ishai Menache, Abhisek Pan, Esaias E Greeff, David Dion, Star Dorminey, Shailesh Joshi, Yang Chen, Mark Russinovich, et al. 2020. Protean:{VM} allocation service at scale. In 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20). 845–861.

[16] Kunal Jain, Anjaly Parayil, Ankur Mallick, Esha Choukse, Xiaoting Qin, Jue Zhang, Íñigo Goiri, Rujia Wang, Chetan Bansal, Victor Rühle, et al. 2024. Intelligent router for llm workloads: Improving performance through workload-aware scheduling. arXiv preprint arXiv:2408.13510 (2024).

[17] Ferdi Kossmann, Bruce Fontaine, Daya Khudia, Michael Cafarella, and Samuel Madden. 2024. Is the GPU Half-Empty or Half-Full? Practical Scheduling Techniques for LLMs. arXiv preprint arXiv:2410.17840 (2024).

[18] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E Gonzalez, et al. 2023. {AlpaServe}: Statistical multiplexing with model parallelism for deep learning serving. In 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23). 663–679.

[19] Hao Liu, Matei Zaharia, and Pieter Abbeel. [n. d.]. RingAttention with Blockwise Transformers for Near-Infinite Context. In The Twelfth International Conference on Learning Representations.

[20] Jiachen Liu, Zhiyu Wu, Jae-Won Chung, Fan Lai, Myungjin Lee, and Mosharaf Chowdhury. 2024. Andes: Defining and Enhancing Quality-of-Experience in LLM-Based Text Streaming Services. arXiv preprint arXiv:2404.16283 (2024).

[21] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. 2024. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. arXiv:2408.06292 [cs.AI] https://arxiv.org/abs/2408.06292

[22] Yixuan Mei, Yonghao Zhuang, Xupeng Miao, Juncheng Yang, Zhihao Jia, and Rashmi Vinayak. 2024. Helix: Distributed Serving of Large Language Models via Max-Flow on Heterogeneous GPUs. arXiv preprint arXiv:2406.01566 (2024).

[23] Xupeng Miao, Chunan Shi, Jiangfei Duan, Xiaoli Xi, Dahua Lin, Bin Cui, and Zhihao Jia. 2024. Spotserve: Serving generative large language models on preemptible instances. In Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2. 1112–1127.

[24] Microsoft. 2024. Online endpoint deployment for real-time inferencing. https://learn.microsoft.com/en-us/azure/machine-learning/concept-endpoints-online.

[25] Microsoft. 2024. Run Azure OpenAI models in batch endpoints to compute embeddings. https://learn.microsoft.com/en-us/azure/machine-learning/how-to-use-batch-model-openai-embeddings.

[26] Chengyi Nie, Rodrigo Fonseca, and Zhenhua Liu. 2024. Aladdin: Joint Placement and Scaling for SLO-Aware LLM Serving. arXiv preprint arXiv:2405.06856 (2024).

[27] OpenAI. 2024. Batch API.

[28] OpenAI. 2024. Streaming API. https://platform.openai.com/docs/api-reference/streaming.

[29] Pratyush Patel, Esha Choukse, Chaojie Zhang, Íñigo Goiri, Aashaka Shah, Saeed Maleki, and Ricardo Bianchini. 2023. Splitwise: Efficient generative llm inference using phase splitting. arXiv preprint arXiv:2311.18677 (2023).

[30] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative llm inference using phase splitting. In 2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA). IEEE, 118–132.

[31] Archit Patke, Dhemath Reddy, Saurabh Jha, Chandra Narayanaswami, Zbigniew Kalbarczyk, and Ravishankar Iyer. 2025. Hierarchical Autoscaling for Large Language Model Serving with Chiron. arXiv:2501.08090 [cs.DC] https://arxiv.org/abs/2501.08090

[32] Yifan Qiao, Shu Anzai, Shan Yu, Haoran Ma, Yang Wang, Miryung Kim, and Harry Xu. 2024. ConServe: Harvesting GPUs for Low-Latency and High-Throughput Large Language Model Serving. arXiv preprint arXiv:2410.01228 (2024).

[33] Nilabja Roy, Abhishek Dubey, and Aniruddha Gokhale. 2011. Efficient autoscaling in the cloud using predictive models for workload forecasting. In 2011 IEEE 4th International Conference on Cloud Computing. IEEE, 500–507.

[34] Krzysztof Rzadca, Pawel Findeisen, Jacek Swiderski, Przemyslaw Zych, Przemyslaw Broniek, Jarek Kusmierek, Pawel Nowak, Beata Strack, Piotr Witusowski, Steven Hand, et al. 2020. Autopilot: workload autoscaling at google. In Proceedings of the Fifteenth European Conference on Computer Systems. 1–16.

[35] P. Schmid, O. Sansevieroa, P. Cuenca, and L. Tunstall. [n. d.]. Llama 2 is here - Get it on Hugging Face [Online]. In Available: https://huggingface.co/blog/llama2.

[36] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E Gonzalez, and Ion Stoica. 2024. Fairness in serving large language models. In 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24). 965–988.

[37] Robert H Shumway, David S Stoffer, Robert H Shumway, and David S Stoffer. 2017. ARIMA models. Time series analysis and its applications: with R examples (2017), 75–163.

[38] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: dynamic scheduling for large language model serving. In Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation (Santa Clara, CA, USA) (OSDI'24). USENIX Association, USA, Article 10, 19 pages.

[39] Prateeksha Varshney and Yogesh Simmhan. 2018. AutoBoT: Resilient and cost-effective scheduling of a bag of tasks on spot VMs. IEEE Transactions on Parallel and Distributed Systems 30, 7 (2018), 1512–1527.

[40] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. 2023. Fast distributed inference serving for large language models. arXiv preprint arXiv:2305.05920 (2023).

[41] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22). USENIX Association, Carlsbad, CA, 521–538. https://www.usenix.org/conference/osdi22/presentation/yu